

Especificación técnica de requisitos

David Muñoz del Valle

Librería extensible de optimización metaheurística en Rust

Tutor

Gabriel Jesús Luque Polo



Índice

Índice	2
1. Alcance y objetivos generales	3
2. Requisitos funcionales	3
RF 2: Implementación de funcionalidades multihilo:	3
RF 3: Definición de Problemas Genéricos:	3
RF 4.1: Sistema de Observabilidad (Telemetría):	4
RF 4.2: Generación de grafos:	4
RF 5: Criterios de Parada Configurables:	4
RF 6: Determinismo y Control de Semillas (Seeding):	4
RF 7: Validación Automática de Modelos:	4
RF 8: Inserción de datos en problemas mediante CSV:	4
RF 9: Serializable:	4
3. Requisitos no funcionales	5
RNF 1: Desarrollo nativo en Rust:	5
RNF 2: Independencia de dependencias:	5
RNF 3: Documentación exhaustiva:	5
RNF 4: Arquitectura modular:	5
RNF 4: Simplicidad de uso:	5
RNF 5: Estabilidad numérica:	5
RNF 6: Eficiencia del software:	5

1. Alcance y objetivos generales

La realización de este proyecto tiene como objetivo el desarrollo de una librería de optimización metaheurística extensible en Rust, diseñada bajo los principios de alto rendimiento y seguridad de memoria. Este proyecto busca paliar la fragmentación y escasez de herramientas maduras en el ecosistema de Rust, proporcionando una base sólida para investigadores y desarrolladores que necesitan una alternativa a los entornos tradicionales en Java, C++ o Python.

Para que podamos estar satisfechos con el resultado de la biblioteca, primero debemos entender cuáles son nuestros objetivos y hasta dónde queremos llegar.

En cuanto a nuestros objetivos principales, se encuentra, obtener un rendimiento, una velocidad de ejecución parecida a las implementaciones líderes de C++, que la biblioteca obtenga la abstracción suficiente como para que se pueda adaptar a los problemas definidos por los usuarios y que sea capaz de realizar mutaciones sobre las poblaciones en paralelo.

También es importante definir lo que está fuera del alcance del proyecto, por ejemplo, no se pretende realizar ningún tipo de interfaz gráfica, todo el funcionamiento se desarrollará mediante línea de comandos.

2. Requisitos funcionales

RF 1: Inclusión de pruebas:

Para asegurar la mantenibilidad y la calidad del código, se desarrollarán pruebas unitarias y de integración.

RF 2: Implementación de funcionalidades multihilo:

Los algoritmos podrán ejecutar sus operadores de manera paralela mediante varios hilos, manteniendo la seguridad en memoria.

RF 3: Definición de Problemas Genéricos:

La biblioteca debe permitir al usuario definir problemas de optimización utilizando cualquier tipo de dato que implemente los traits requeridos. La biblioteca también deberá implementar por defecto varios problemas para que sirvan como ejemplos.

RF 4.1: Sistema de Observabilidad (Telemetría):

La biblioteca debe permitir inyectar "Observadores" para monitorizar en tiempo real la convergencia, la velocidad y la calidad de las soluciones obtenidas por los algoritmos.

RF 4.2: Generación de grafos:

El sistema de observabilidad, debe poder crear gráficos fácilmente interpretables, que muestren la calidad de las soluciones obtenidas y la evolución de esta misma con cada iteración de los operadores durante la ejecución del algoritmo.

RF 5: Criterios de Parada Configurables:

El usuario debe poder detener la ejecución por tiempo, número de iteraciones o nivel de precisión alcanzado.

RF 6: Determinismo y Control de Semillas (Seeding):

Debido a la naturaleza de los algoritmos heurísticos, es necesario introducir cierto grado de aleatoriedad en su ejecución. En este sistema, dicha aleatoriedad se implementa mediante generación pseudoaleatoria.

Con el fin de garantizar la reproducibilidad de los resultados, el sistema deberá permitir al usuario definir una semilla (seed) para el generador de números pseudoaleatorios. De este modo, utilizando la misma semilla y las mismas condiciones de entrada, el algoritmo producirá siempre los mismos resultados, facilitando así la validación, comparación y análisis experimental.

RF 7: Validación Automática de Modelos:

El sistema debe poder verificar antes de ejecutar si el algoritmo está bien definido, teniendo en cuenta las variables y las restricciones definidas.

RF 8: Inserción de datos en problemas mediante CSV:

Los algoritmos podrán importar variables, restricciones y otros conjuntos de datos desde archivos CSV, lo que facilitará la definición y optimización de problemas complejos mediante el uso de adaptadores compatibles con este formato.

RF 9: Serializable:

Todas las estructuras deben poder implementar los traits de *serde*, para que estas sean serializables.

3. Requisitos no funcionales

RNF 1: Desarrollo nativo en Rust:

La biblioteca debe estar desarrollada 100% en el lenguaje de programación Rust.

RNF 2: Independencia de dependencias:

La biblioteca no debe depender de crates externos (fuera de la estándar de Rust), *Zero-dependency policy*.

RNF 3: Documentación exhaustiva:

El código debe tener una documentación clara y concisa.

RNF 4: Arquitectura modular:

El diseño debe separar claramente los componentes del sistema, permitiendo intercambiarlos sin modificar el núcleo.

RNF 4: Simplicidad de uso:

Debe ser fácil e intuitiva la incorporación de nuevos problemas por parte del usuario.

RNF 5: Estabilidad numérica:

El sistema debe ser capaz de solventar los problemas de redondeo causados por los tipos decimales f32 y f64.

RNF 6: Eficiencia del software:

La biblioteca final debe ser capaz de realizar sus funcionalidades con una velocidad más que notable.